

Universal IT Project Development Protocol (v1.0)

Advanced Blueprint for AI-Assisted, Cloud-Native Full-Stack Development
Integrating PMBOK®, Agile, and Strict Multi-Agent Guardrails

1.0 Executive Overview

This framework governs the end-to-end lifecycle of modern IT infrastructure. Upgraded for the AI and Cloud-Native era, this protocol addresses the core failure modes of autonomous "vibe coding" (e.g., specification drift, premature implementation, and security vulnerabilities). It establishes a rigid boundary between planning, architecture, and autonomous code execution, ensuring enterprise-grade security and alignment with business objectives.

2.0 Project Roadmap & Release Strategy

2.1 Roadmap Hierarchy (PMBOK Aligned)

- Themes: High-level strategic objectives (e.g., "AI Pipeline Integration").
- Epics: Large bodies of deliverable work.
- Features: Specific functionalities providing immediate user value.

2.2 The MoSCoW Release Strategy

Scope must be rigidly locked per sprint to prevent AI hallucination and scope creep. Define Must-Haves, Should-Haves, Could-Haves, and Won't-Haves before any code is generated.

3.0 Security, Compliance, and Cloud-Native Architecture

To guarantee safety in full-stack LLM applications, autonomous agents must adhere to strict zero-trust principles.

3.1 Full-Stack Safety Parameters

- Secret Management: AI agents must NEVER hardcode API keys or credentials. All secrets must be injected via environment variables (.env).
- Data Sanitization: All inputs handled by the backend must be sanitized to prevent prompt injection, SQL injection, and XSS.
- Compliance Auditing: Code generated by AI must pass automated SAST (Static Application Security Testing) before deployment.

4.0 The Structured Multi-Agent AI Workflow

Unstructured AI coding leads to conflated roles and tangled architecture. This protocol enforces a strict separation of concerns among actors:

4.1 Actor Separation

1. The Human (Product Owner): Provides the business case, approves designs, and tests the final product.
2. The Architect LLM (Conversational): Consumes requirements and produces a durable Technical Specification (Architecture.md). IT DOES NOT WRITE APP CODE.
3. The Coding Agent (Executor): Reads the Technical Specification and writes code strictly adhering to it. IT DOES NOT INVENT FEATURES.

4.2 Design Freeze Mechanism

Before the Coding Agent is invoked, the Architect LLM's specification must undergo a "Design Freeze." If the human changes requirements mid-development, the change must be updated in the Specification Document first, NOT directly in the code.

5.0 The "Vibe Coding" Execution Loop

The Coding Agent must execute the following state-machine loop for every feature:

1. Ingest Context: Read the frozen Architecture.md and Business Case.
2. Phase-Close Testing (TDD): Scaffold the unit tests for the expected behavior first.
3. Execute: Implement the feature until the test passes. Limit file sizes (Modularization).
4. Error Handling: If an error occurs, output the log, state the hypothesis, and apply a targeted fix. No blind guessing.
5. Document & Persist: Update CHANGELOG.md to ensure the agent maintains state memory across context window resets.

6.0 Success Metrics & Continuous Delivery

6.1 Engineering Metrics

- Defect Density: Bugs per 1k lines of AI-generated code.
- Pipeline Automation: Time to deploy via CI/CD pipelines (e.g., Docker/Kubernetes in the cloud).

6.2 Business & Outcome Metrics

- Target MRR / User Adoption: As defined in the Business Case.
- Application Health: SLA uptime, API latency, and real-time compliance status.